CSC 345 Report for Project 2
Full Name: Paris Watznauer, Damaka Morisseau, and Anthony Carvajal
Section: 02

Breakdown of main.c:

Section 1.1: Checking a Region: Source Code

```c
/*1 thread checks all rows*/
void* validate_all_rows(void* param) {
    for (int row = 0; row < 9; row ++) {
        int marked[10] = {0};    /*/rray marking seen values*/

        for (int column = 0; column < 9; column++) {
            int value = puzzle[row][column];

            if (value < 1 || value > 9 || marked[value] == 1) {
                valid1[0] = 0;
                pthread_exit(NULL); /* signifies value was already seen in row, so puzzle is invalid*/
            }

            marked[value] = 1;    /*mark array index corresponding to seen value to signify already seen in row*/
        }
    }

    valid1[0] = 1;  /*mark rows as valid*/
    pthread_exit(NULL);
}
```

Explanation: Every worker thread checks if it's part of the board in the same manner. It uses an array titled "marked" with 10 slots, all starting at 0. For each cell, it first checks that the value is between 1 and 9, then it checks whether the marked value in the array is already 1, which is effectively a check to see if this value has already shown up. If either of these checks fails, the region is deemed invalid, so the thread writes a 0 to its result slot and exits afterward via pthread_exit. But if the value passes, the thread sets the marked value to 1 and simply moves on.

If the thread makes it through all of the nine cells without issue, in this case meaning the region has every digit from 1-9 exactly once, it writes a 1. The source code above is the thread that checks all nine rows in mode 1. The

same idea is used for the column, single row, single column, and subgrid functions and only changes what cells are looped over.

Section 2.1: Passing Parameters and Returning Results: Source Code

```
8   typedef struct {
9       int row;
10      int column;
11      int index;
12  } parameters;
13
14  /*1 threads checking all rows and columns + 9 threads each checking a subgrid
15  row checker writes to val[0], colunm to val[1] and subgrid to rest
16  if array is all 1, puzzle is valid, if any zeroes then a validate function has failed to validate and puzzle is invalid*/
17  int valid1[11] = {0};
18
19  /*for 27 thread implementation (option 2)*/
20  int valid2[27] = {0};
```

```
for (int i = 0; i < 11; i++) {
    if (valid1[i] == 0) {
        valid = 0;
        break;
    }
}
```

Explanation: Using the assignment suggestion, we opted to use a struct named "parameters" which would tell each thread what part of the board to check. The struct index tells the thread which slot in the result array belongs to it. Row and column give the thread the location it should check, but how they are used depends on the thread itself.

As for the results, we used two global arrays: valid1 with 11 slots for mode 1 and valid with 27 slots for mode 2. Each thread writes a 1 to its own slot if its region is valid and 0 if invalid. After all the threads finish, main checks every slot, and if any of them is 0, the board is not a valid solution. We did not need to implement a mutex because each thread only ever writes to its own slot. Furthermore, main does not read the array until every thread has been joined, so two threads are never able to write to the same block of memory.

Section 3.1: Mode 1 (11 threads): Source Code

```
if (mode == 1) {    /*mode 1 11 threads*/
    pthread_t threads[11];
    parameters data[11];

    for (int i = 0; i < 11; i++) {
        if (i == 0) {
            pthread_create(&threads[i], NULL, validate_all_rows, NULL);
        } else if (i == 1){
            pthread_create(&threads[i], NULL, validate_all_columns, NULL);
        } else{
            data[i].row = ((i - 2) / 3) * 3;
            data[i].column = ((i - 2) % 3) * 3;
            data[i].index = i;
            pthread_create(&threads[i], NULL, validateSub, &data[i]);

        }
    }

    for (int i = 0; i < 11; i++){
        pthread_join(threads[i], NULL);
    }
```

Explanation: We constructed Mode 1 to follow the first strategy from the assignment. Thread 0 runs validate_all_rows to check all nine rows, and thread 1 runs validate_all_columns to check all nine columns. Threads 2-10 each run validateSub on one of the nine 3x3 subgrids. To give each subgrid thread its starting position, we calculated the top-left corner from the loop counter by using ((i - 2) / 3) * 3; this gives the row, and ((i - 2) % 3) * 3 gives the column. This walks through the subgrids in order. Each thread gets its own entry in the "data" array, so no two threads are allowed to share parameters. After creating all 11 of these threads, main waits for each one with pthread_join before checking the final results.

Section 4.1: Mode 2 (27 threads): Source Code

```c
else {  /*Mode 2 27 threads*/
    pthread_t threads[27];
    parameters data[27];
    int thread_count = 0;

    for (int r = 0; r < 9; r++) {
        data[thread_count].row = r;
        data[thread_count].column = 0;
        data[thread_count].index = thread_count;
        pthread_create(&threads[thread_count], NULL, validate_single_row, &data[thread_count]);
        thread_count++;
    }

     for (int c = 0; c < 9; c++) {
        data[thread_count].row = 0;
        data[thread_count].column = c;
        data[thread_count].index = thread_count;
        pthread_create(&threads[thread_count], NULL, validate_single_column, &data[thread_count]);
        thread_count++;
    }

    for (int r = 0; r < 9; r += 3) {
        for (int c = 0; c < 9; c += 3) {
            data[thread_count].row = r;
            data[thread_count].column = c;
            data[thread_count].index = thread_count;
            pthread_create(&threads[thread_count], NULL, validateSub, &data[thread_count]);
            thread_count++;
        }
    }

     for (int i = 0; i < 27; i++){
        pthread_join(threads[i], NULL);
    }

    for (int i = 0; i < 27; i++) {
        if (valid2[i] == 0) {
            valid = 0;
            break;
        }
    }
}
```

Section 4.2: Output

```
trinkendarf@Anthonys-MacBook-Pro-47 CSC345-Project2 % ./main 2
BOARD STATE IN input.txt:
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: YES (0.000701 seconds)
```

Explanation: Mode 2 is the second argument option we are told to implement. Instead of one thread checking every row, we instead utilize nine threads to each check a single row using validate_single_row, and nine threads each check a single column using validate_single_column. The last nine threads check the subgrids comparable to what we did in mode 1. We opted to reuse validateSub to reduce redundancy in the code. Since this subgrid function needs to write to valid1 and valid2 in their respective modes, it checks its index. Mode 1 subgrid threads use indices 2 to 10, and mode 2 uses 18 to 26. So an index below 11 always means mode 1. The output is the same as mode 1 except it has a longer time due to needing to create many more threads.

Section 5.1: Monitoring Time: Source Code

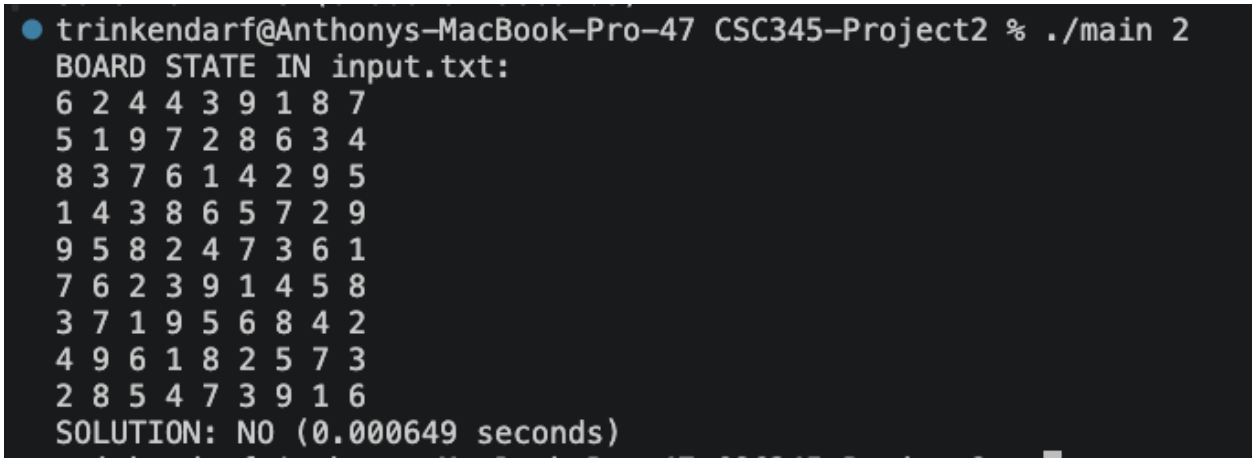
```
185   struct timespec start, end;
186
187   int valid = 1;
188
189   clock_gettime(CLOCK_MONOTONIC, &start);
190
191   if (mode == 1) {    /*mode 1 11 threads*/
192       pthread_t threads[11];
193       parameters data[11];
194
195       for (int i = 0; i < 11; i++) {
196           if (i == 0) {
197               pthread_create(&threads[i], NULL, validate_all_rows, NULL);
198           } else if (i == 1){
199               pthread_create(&threads[i], NULL, validate_all_columns, NULL);
200           } else{
201               data[i].row = ((i - 2) / 3) * 3;
202               data[i].column = ((i - 2) % 3) * 3;
203               data[i].index = i;
204               pthread_create(&threads[i], NULL, validateSub, &data[i]);
205
206           }
207       }
208
209       for (int i = 0; i < 11; i++){
210           pthread_join(threads[i], NULL);
211       }
212
213       for (int i = 0; i < 11; i++) {
214           if (valid1[i] == 0) {
215               valid = 0;
216               break;
217           }
218       }
219   }
```

```
264   clock_gettime(CLOCK_MONOTONIC, &end);
265
266   double elapsed_time = (end.tv_sec - start.tv_sec) + (end.tv_nsec - start.tv_nsec) / 1e9;
267
268   if (valid) {
269       printf("SOLUTION: YES (%.6f seconds)\n", elapsed_time);
270   } else {
271       printf("SOLUTION: NO (%.6f seconds)\n", elapsed_time);
272   }
273
274   return 0;
275   }
```

Explanation: In order to measure how long each mode takes, we used clock_gettime with CLOCK_MONOTONIC, which is not affected by the system clock. Without this standard, the system clock could affect our final time output. Essentially, the timer starts right before the threads are created, and then stops after every thread has been joined and the result has been checked. This is necessary as we are specifically tasked with comparing the total run time of the two methods. Both modes do the same total amount of checking, so the cost of the threads themselves is the true differentiator. Elapsed time is then printed in seconds next to a YES or NO.

Section 6.1: Invalid Board: Output

```
trinkendarf@Anthonys-MacBook-Pro-47 CSC345-Project2 % ./main 1
BOARD STATE IN input.txt:
6 2 4 4 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: NO (0.000455 seconds)
```



Explanation: This is a check to ensure that a program does not output YES every time and is actually functional when it comes to checking errors in the input text. To do this, we simply changed the 5 in the first row to a 4, and it broke the board in 3 ways:

1. The first row now has two 4s
2. The fourth column now has two 4s
3. The top middle subgrid also has two 4s

All three are also missing a 5. The program functioned as intended and printed NO in both modes, as shown in the output above. In mode 1, the all-rows thread, the all-columns thread, and the thread for the top middle subgrid all write a 0 to their result slots. As for mode 2, the threads for row 1, column 4, and the top middle subgrid also write a 0 to their slots. Therefore, since at least one slot is 0, main reports that the board is not a solution, which is the desired behavior.

Section 7.1: Reading Input and Printing Board: Source Code

```c
FILE *file = fopen("input.txt", "r");
if (file == NULL) {
    perror("Error opening input file");
    return 1;
}

for (int row = 0; row < 9; row++){
    for (int column = 0; column < 9; column++) {
        if (fscanf(file, "%d", &puzzle[row][column]) != 1) {
            printf("Error reading input file\n");
            fclose(file);
            return 1;
        }
    }
}
fclose(file);

printf("BOARD STATE IN input.txt:\n");
for (int r = 0; r < 9; r++) {
    for (int c = 0; c < 9; c++) {
        printf("%d%s", puzzle[r][c], (c == 8) ? "" : " ");
    }
    printf("\n");
}
```

Section 7.2: Output

```
BOARD STATE IN input.txt:
6 2 4 5 3 9 1 8 7
5 1 9 7 2 8 6 3 4
8 3 7 6 1 4 2 9 5
1 4 3 8 6 5 7 2 9
9 5 8 2 4 7 3 6 1
7 6 2 3 9 1 4 5 8
3 7 1 9 5 6 8 4 2
4 9 6 1 8 2 5 7 3
2 8 5 4 7 3 9 1 6
SOLUTION: YES (0.000335 seconds)
```

Explanation: Our program was set up to first check if it was given exactly one argument, this being 1 or 2. This argument decides what threading mode the program executes under. It then opens our Input.txt and reads the 81 numbers into the global 9x9 array under the name "puzzle". This is done using fscanf, which was implemented in a nested loop. Once the board is read in, the program prints out the desired format of the Sudoku puzzle. If the file can not be opened or has an insufficient number of entries, then the program exits. The puzzle was made to be global so every thread would be able to read it without it being passed as a copy.

Breakdown of Experimental Approach:

Section 1.1: Source Code

experiment.py > ...

```python
#Statistical experiment comparing mode 1 (11 threads) and mode 2 (27 threads).
#Null hypothesis: there is no statistically significant difference between the two methods!

import subprocess  #lets Python run ./main like we would in the terminal
import statistics  #mean and standard dev
import matplotlib.pyplot as plt  #For drawing chart
from scipy import stats #for t-test

RUNS = 50  #number of times to run each mode


def get_time(mode):
    #runs ./main and pulls the time out of the last line,
    #"SOLUTION: YES (0.000184 seconds)" -> 0.000184
    output = subprocess.run(["./main", str(mode)], capture_output=True, text=True).stdout
    last_line = output.strip().split("\n")[-1] #the "SOLUTION: ..." line
    return float(last_line.split("(")[1].split(" ")[0])  #text after "(" and before "seconds".


#Lists to hold the run time of every single run
mode1_times = []
mode2_times = []

#Alternate between modes so both get the same conditions on the machine
for i in range(RUNS):
    mode1_times.append(get_time(1) * 1000)  #convert seconds to milliseconds
    mode2_times.append(get_time(2) * 1000)

#print the average and standard dev for each mode
print("Runs per mode:", RUNS)

print(f"Mode 1 (11 threads): mean = {statistics.mean(mode1_times):.4f} ms, std dev = {statistics.stdev(mode1_times):.4f} ms")
print(f"Mode 2 (27 threads): mean = {statistics.mean(mode2_times):.4f} ms, std dev = {statistics.stdev(mode2_times):.4f} ms")

#Welch's t-test (doesn't assume both modes have the same variance)
#The p-value is the chance of seeing a difference this big if the two modes were actually the same
t_stat, p_value = stats.ttest_ind(mode1_times, mode2_times, equal_var=False)
print(f"t = {t_stat:.3f}, p-value = {p_value:.3g}")

#Compare the p-value to our significance level of 0.05 to accept or reject the null hypothesis
if p_value < 0.05:
    print("p < 0.05, so we reject the null hypothesis: there is a significant difference.")
else:
    print("p >= 0.05, so we fail to reject the null hypothesis: no significant difference.")

#Bar chart of the average runn time for each mode (error bars = standard dev)
plt.bar(["Mode 1 (11 threads)", "Mode 2 (27 threads)"],
        [statistics.mean(mode1_times), statistics.mean(mode2_times)],
        yerr=[statistics.stdev(mode1_times), statistics.stdev(mode2_times)],
        capsize=10)  #width of the little caps on the error bars

plt.ylabel("Average run time (ms)")

plt.title(f"Sudoku validator average run time ({RUNS} runs per mode)")

#Save the chart as an image in the same folder
plt.savefig("results.png")

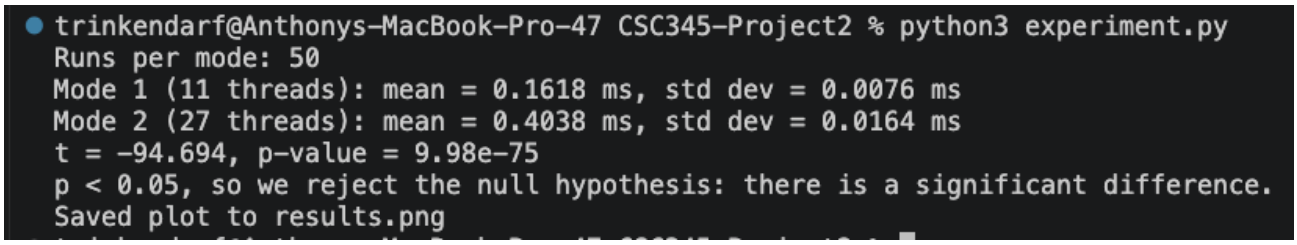
print("Saved plot to results.png")

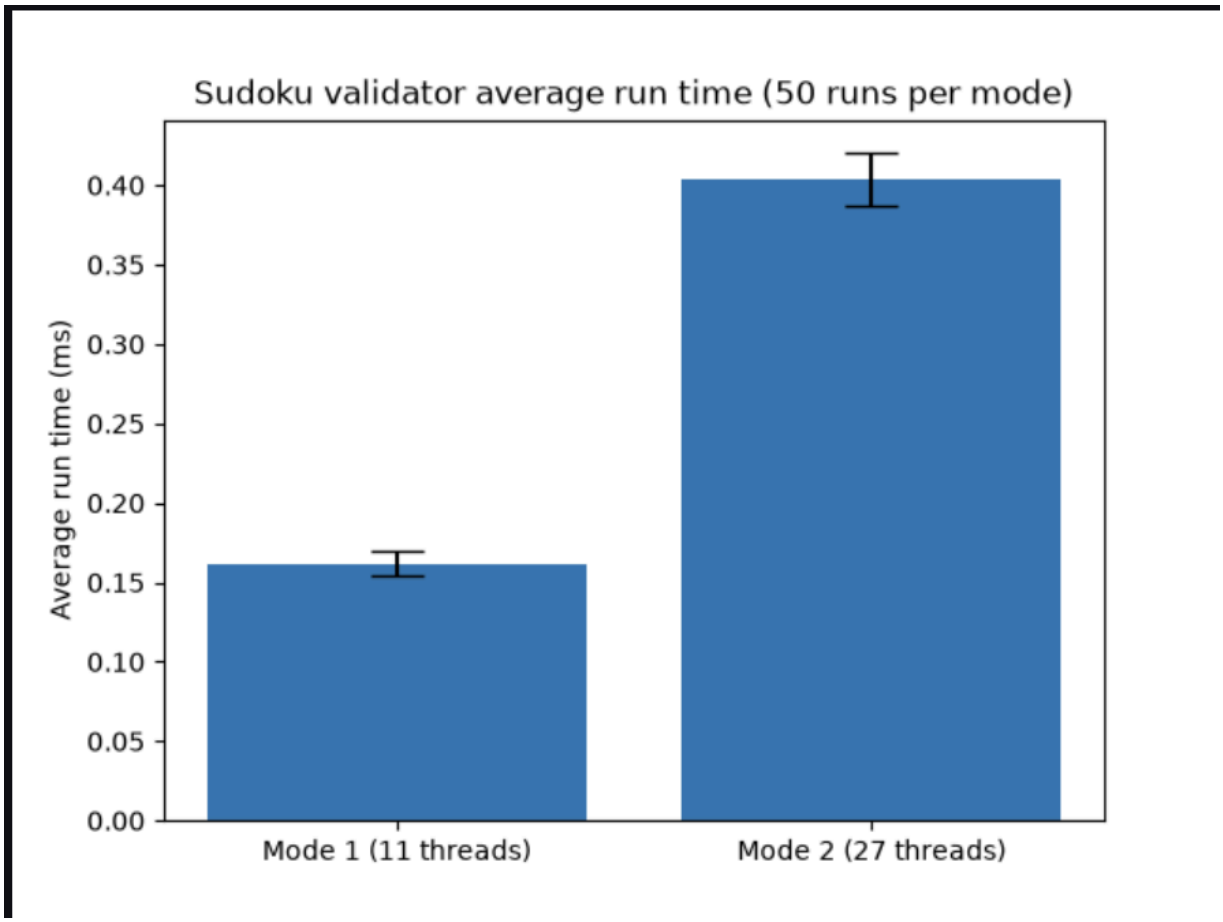
```

Explanation: To compare the two modes, we opted to write a Python script. Our null hypothesis that was decided on was that there is no statistically significant difference between mode 1 and mode 2, and the level of significance used was 0.05. The script uses "subprocess" to run both variants of program execution 50 times each; this is the equivalent of just typing the command 50 times each in the terminal.

It alternates between the two modes, so if the computer is busy with something else, both modes are impacted equally, ensuring fairness. For each run, the get_time function takes the last line of the output and pulls out the number of seconds. Once all the runs are finished, the script calculates the mean, standard deviation, and then runs a Welch's t-test. We used this test since it aligns with our methodology of not assuming that the two modes have the same variance. The script compares the calculated p-value to 0.05 and prints whether or not we reject the null hypothesis. Finally, it has the added functionality of forming a bar chart of the average time for each of the modes, accompanied by error bars displaying the standard deviation.

Statistical Experiment Results (Requirement 3):





We ran the experiment on a 2023 MacBook Pro with an Apple M2 Pro chip, which has 10 cores in total. Mode 1 averaged .1618 ms with a standard deviation of .0076 ms, and Mode 2 had an average of .4038 ms with a standard deviation of .0164 ms. When looking at our p-value from the t-test, we came to our conclusion. The null hypothesis is rejected. The calculated p-value is 9.98 * 10^-75, which is quite a bit below the significance level of 0.05. Meaning there is a statistically significant difference between the two methods. Mode 1 is faster.

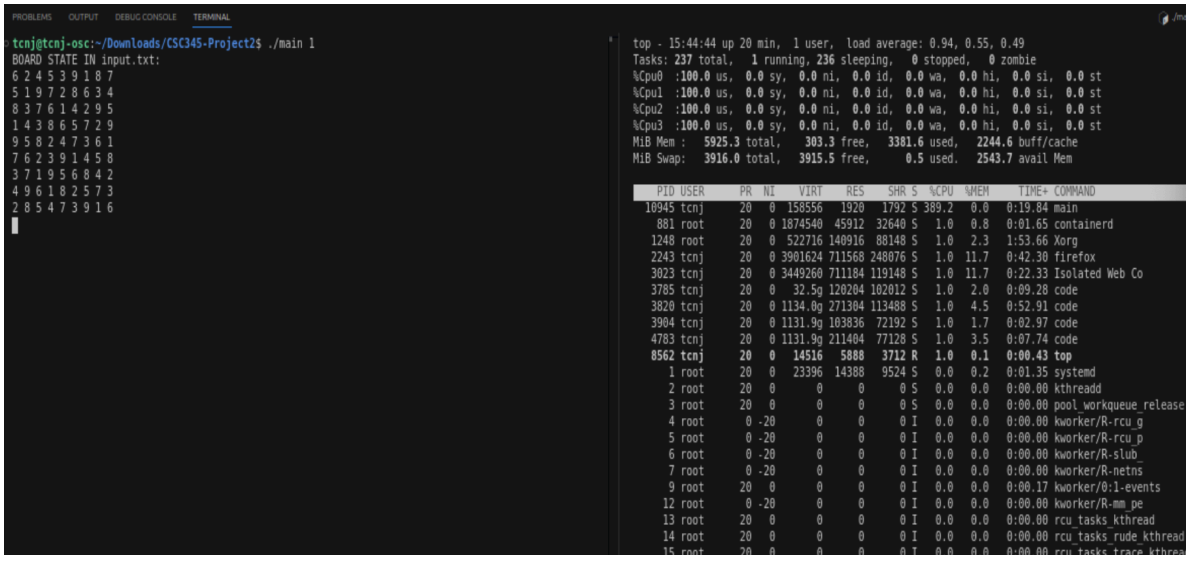
Before we began, we were under the impression that more threads would be faster since they would be able to split up the jobs into smaller pieces. But we found out it was quite the opposite. The key points we extrapolated from the output are that mode 1 is more than twice as fast as mode 2 (0.4038 / 0.1618 ≈ 2.5). Furthermore, the fact that this was run 50

times is meant to solidify that the results are not a one-off instance of a mode beating out another by coincidence. Both standard deviations are small compared to the means. Each mode took nearly the same time every run; there were very few spikes during the runs as a whole, pointing towards consistency. Almost *every* mode 1 run beats out mode 2 runs. Finally, using mode averages per thread, we can gather that time is by and large thread overhead. Mode 1 averages .1618 / 11 (thread count) ≈ 0.0147 and mode 2 averages .4038 / 27 (thread count) ≈ 0.0150 ms per thread. The thread cost is very similar, so the total time grows with the number of threads being created. To loop back to the concept of mode 1 being a bit more than twice as fast as mode 2, mode 2 has roughly 2.45 times as many threads and takes about 2.5 times as long. The ratios match nicely. This difference in performance is no coincidence.

The idea that more threads result in a faster calculation is often true, but *only* when the work is complex enough to make up for the time of thread creation. Checking a 9x9 board is so simple that 27 threads become overkill, and the execution time ends up suffering due to extra thread overhead.

Is The Program Using Multiple Cores? (Requirement 4):

Evidence:



Prep:

To be able to prove whether or not the program uses multiple cores, we needed to make 2 additions to our main.c code. The problem we were faced with was that, when executing a normal run, it would be done in less than a second. Each thread finishes almost immediately, so the program is done way before a tool like top can actually display to us what it's doing. We needed threads to run long enough in order to observe whether or not our program uses multiple cores. To solve this, we added a function called busy_workDeluxe that runs an empty loop 10 billion times, and called it at the start of validateSub. Since validateSub is used by the 9 subgrid threads, this tiny addition keeps those 9 threads busy at the same time. We covered this before, but modern compilers are too smart and sometimes will skip code, so the variable had to be marked volatile so it would execute every time. After testing, these two additions are commented out but are still present if observation is needed again.

Results & Conclusion:

This example run was done on our virtual machine. Much like one of our prior labs, we opened two terminals side by side. In one, we ran top with 1 so we could see each core on its own line and how much CPU it was using. In the other, we just ran the main program.

As shown in the image above, the program was running all four cores (the rows of CPU0 through CPU3) at 100% user time at the same time. Furthermore, our main program, which you can see at PID 10945 (the first row under the white bar), was at the top of the process list with a total CPU usage of 389.2%. Each core counts as 100%, so a total of 389% means the program was almost utilizing 4 cores at once. Nothing else within the system was taking significant amounts of the CPU, so we can assume that its usage is from our program.

This answers the question posed to us. The program does run on multiple cores in parallel. In this specific instance, since mode 1, for example, has 9 subgrid threads, which is more than the 4 available cores, Linux spreads them across the cores to keep them all busy. So **yes**, our program runs on multiple cores in parallel.